

Olist E-Commerce Analytics

Milestone 2: Jinja, Macros & Packages

dbt + BigQuery Portfolio Project

Soichiro Tanabe
June 2026

Table of Contents

1. EXECUTIVE SUMMARY	2
2. BUSINESS SCENARIO	2
3. OBJECTIVES	2
4. PROJECT STRUCTURE	3
5. MACRO IMPLEMENTATIONS	4
5.1 CENTS_TO_DOLLARS — PARAMETERISED VALUE TRANSFORMATION	4
5.2 CLEAN_STRING — REUSABLE TEXT STANDARDISATION	4
5.3 COALESCE_COLUMNS — JINJA FOR-LOOP	4
5.4 DESIGN DECISION: GENERATE_SCHEMA_NAME OVERRIDE EXCLUDED	4
6. KEY LEARNINGS	5
7. PROJECT COMPLETION SUMMARY	5
<i>Related Work</i>	<i>5</i>

1. Executive Summary

This report documents Milestone 2 of the Olist E-Commerce Analytics portfolio project, extending the original dbt and Google BigQuery pipeline built in Milestone 1. While Milestone 1 focused on core analytics engineering fundamentals — staging models, mart models, testing, and documentation — Milestone 2 builds directly on that foundation to demonstrate intermediate dbt development skills: Jinja templating, custom macros, and third-party dbt packages.

The work follows the dbt Labs “Advanced Jinja and Macros” course material and applies each concept to the existing Olist dataset rather than working through isolated exercises. This approach was deliberately chosen to keep the portfolio cohesive: every macro and package introduced in this milestone is applied to real models from the original project, producing a single continuous body of work rather than disconnected practice files.

Key additions in this milestone include: installation and use of the `dbt_utils` and `dbt_expectations` packages; and four custom Jinja macros covering value transformation, string cleaning, conditional coalescing, and environment-aware filtering. These macros are applied directly to existing staging and mart models from Milestone 1. Additionally, a new `fct_reviews` model has been fully designed and documented, ready for execution once the development environment issue described in this report is resolved.

This milestone also involved a substantial environment troubleshooting exercise, detailed in the Key Learnings section, which reflects the kind of operational debugging real analytics engineers regularly encounter when working with cloud-based development environments.

2. Business Scenario

The business context remains the same as Milestone 1: Olist, a Brazilian e-commerce platform connecting merchants with customers across multiple marketplaces. Milestone 1 established the foundational data model needed to answer high-level business questions about revenue, sellers, and customer location.

Milestone 2 does not introduce new business questions. Instead, it focuses on improving the engineering quality of the existing transformation layer — applying reusable Jinja macros to models already built in Milestone 1, and installing community packages that extend dbt's built-in capabilities.

This mirrors how real analytics engineering teams work: after an initial data model is established, the focus shifts to maintainability, consistency, and reducing duplicated SQL logic before expanding the model further.

3. Objectives

The objectives for this milestone were:

- Install and configure dbt packages (`dbt_utils`, `dbt_expectations`) via `packages.yml`
- Develop reusable Jinja macros to demonstrate variables, conditionals, loops, and the target context object
- Apply each macro to existing staging and mart models from Milestone 1, rather than building isolated examples

- Use package-provided macros (`dbt_utils.generate_surrogate_key`) alongside custom macros within the same model
- Document new macros and the new model in `schema.yml`, consistent with Milestone 1 documentation standards
- Push all changes to the existing GitHub repository, maintaining a single continuous project history

4. Project Structure

The project structure builds directly on the Milestone 1 layout, with two key additions: a `packages.yml` file at the project root, and new files within the existing `macros/` and `models/marts/` directories.

```
olist_portfolio/
├── macros/
│   ├── cents_to_dollars.sql      (new)
│   ├── clean_string.sql         (new)
│   ├── coalesce_columns.sql     (new)
│   └── schema.yml               (new)
├── models/
│   ├── staging/
│   │   ├── sources.yml
│   │   ├── schema.yml
│   │   ├── stg_orders.sql
│   │   ├── stg_customers.sql    (updated – uses clean_string)
│   │   ├── stg_order_items.sql
│   │   ├── stg_order_payments.sql
│   │   ├── stg_order_reviews.sql
│   │   ├── stg_products.sql
│   │   └── stg_sellers.sql
│   └── marts/
│       ├── schema.yml           (updated)
│       ├── fct_orders.sql       (updated)
│       ├── dim_customers.sql
│       ├── dim_products.sql
│       └── dim_sellers.sql      (updated)
├── dbt_project.yml
├── packages.yml                (new)
└── README.md
```

`packages.yml` was added to the project root, declaring two community packages used throughout this milestone:

```
packages:
- package: dbt-labs/dbt_utils
  version: [">=1.0.0", "<2.0.0"]
- package: calogica/dbt_expectations
  version: [">=0.10.0", "<1.0.0"]
```

Running `dbt deps` downloads these packages into a local `dbt_packages/` directory, which is excluded from version control via `.gitignore`, consistent with dbt community standards.

5. Macro Implementations

Four custom Jinja macros were developed for this milestone. Each was selected to demonstrate a distinct Jinja capability covered in the dbt Labs Advanced Jinja and Macros course, and each is applied to a real model in the project rather than left as a standalone exercise.

5.1 cents_to_dollars — Parameterised value transformation

Demonstrates macro arguments with a default value. Converts an integer cents column into a rounded decimal value, with configurable rounding precision.

```
{% macro cents_to_dollars(column_name, decimal_places=2) %}
    round({{ column_name }} / 100, {{ decimal_places }})
{% endmacro %}
```

5.2 clean_string — Reusable text standardisation

A simple but widely reused macro applied across multiple staging models to standardise text fields (city and state names) by lowercasing and trimming whitespace, reducing duplicated SQL logic across models.

```
{% macro clean_string(column_name) %}
    lower(trim({{ column_name }}))
{% endmacro %}

-- Applied in stg_customers.sql:
{{ clean_string('customer_city') }} as customer_city,
{{ clean_string('customer_state') }} as customer_state
```

5.3 coalesce_columns — Jinja for-loop

Demonstrates a Jinja for-loop with conditional comma placement, generating a dynamic COALESCE expression across an arbitrary list of columns. This avoids manually writing COALESCE statements when the number of input columns may change.

```
{% macro coalesce_columns(columns, default=0) %}
    coalesce(
        {% for col in columns %}
            {{ col }}{% if not loop.last %}, {% endif %}
        {% endfor %}
        , {{ default }}
    )
{% endmacro %}
```

5.4 Design decision: generate_schema_name override excluded

An override of dbt's built-in generate_schema_name macro was considered but deliberately excluded from this milestone. The project uses a single target schema (dbt_olist) with no multi-schema separation between staging and marts, so the macro would have no observable effect on the build output. It is also a macro almost universally copied verbatim from dbt documentation rather than authored, and overriding a built-in macro unnecessarily introduces risk without a corresponding portfolio benefit. This decision itself is included here as a demonstration of judgement in scoping technical work appropriately.

6. Key Learnings

- Gained practical experience with dbt packages, including installing third-party packages via `packages.yml` and using package-provided macros (`dbt_utils.generate_surrogate_key`) alongside custom project macros within the same model
- Developed four original Jinja macros covering parameterised arguments with defaults, for-loops, and conditional logic using the target context object — moving beyond the built-in `source()` and `ref()` macros used in Milestone 1
- Understood macro authoring trade-offs: deliberately chose not to override a built-in dbt macro (`generate_schema_name`) where it added project risk without measurable benefit, reinforcing the importance of applying techniques where they genuinely fit the project rather than for their own sake

7. Project Completion Summary

The following milestone tasks were completed:

- ✓ Installed `dbt_utils` and `dbt_expectations` packages via `packages.yml`
- ✓ Successfully ran `dbt deps` and confirmed package installation (`dbt_packages/` generated)
- ✓ Authored four custom Jinja macros: `cents_to_dollars`, `clean_string`, `coalesce_columns`, `limit_data_in_dev`
- ✓ Applied `clean_string` within an existing staging model (`stg_customers`)
- ✓ Documented the new macros and model, extending the existing `schema.yml` documentation standard
- ✓ Made and documented a deliberate scoping decision to exclude an unnecessary macro override
- ✓ Diagnosed and documented a dbt Cloud IDE connectivity issue through a structured, multi-layer troubleshooting process
- ✓ Committed and pushed all Milestone 2 changes to the existing GitHub repository

Related Work

This milestone builds directly on Milestone 1 of this project. The original repository, including the full staging and mart layer, data quality tests, and documentation referenced throughout this report, is available at:

github.com/soichi-berson/dbt_olist_milestone1